

MedLicense

Technical Documentation

Generated May 27, 2026

MedLicense — Technical Documentation

Rwanda's professional medical license exam preparation platform.

Table of Contents

1. [Project Overview](#)
 2. [Tech Stack](#)
 3. [Project Structure](#)
 4. [Environment Setup](#)
 5. [Database Schema](#)
 6. [Authentication](#)
 7. [API Reference](#)
 8. [Pages & Routing](#)
 9. [Core Features](#)
 10. [Admin Panel](#)
 11. [Exam Engine](#)
 12. [Localization \(EN/FR\)](#)
 13. [Payments & Subscriptions](#)
 14. [Video Tutorials \(Mux\)](#)
 15. [Notifications](#)
 16. [License Categories](#)
 17. [Components Reference](#)
 18. [Deployment](#)
-

1. Project Overview

MedLicense is a full-stack web application that prepares healthcare professionals in Rwanda for their national licensing exams. The platform supports 16 different medical license types across 5 professional groups.

Core Value:

- Practice with exam simulations that mirror the real Rwanda medical licensing exams
- Study in English or French
- Track performance over time with detailed analytics

- Access video tutorials by subject area
- Unlock premium content via mobile money or card payment

User Types:

Role	Access
STUDENT	Dashboard, exams, results, analytics, saved questions, subscription
ADMIN	Everything above + full admin panel (questions, exams, users, videos, analytics, payments)

2. Tech Stack

Framework

Technology	Version	Purpose
Next.js	16.2.6	Full-stack React framework (App Router, Turbopack)
React	19.2.4	UI rendering
TypeScript	5.x	Type safety

Backend & Data

Technology	Version	Purpose
Supabase	^2.49.0	PostgreSQL database + auth helpers
NextAuth	5.0.0-beta.31	Authentication (JWT strategy)
bcryptjs	—	Password hashing
Zod	—	Schema validation

Styling

Technology	Version	Purpose
Tailwind CSS	v4	Utility-first CSS
Lucide React	—	Icon library
Framer Motion	—	Animations

External Services

Service	Purpose
Mux	Video upload, processing & streaming
Afripay	Payment gateway (MoMo, Card, Afripay)
Nodemailer + SMTP	Transactional emails (verification, password reset)
Google OAuth	Social login

Dev Tools

Tool	Purpose
pnpm	Package manager
ESLint 9	Linting
tsx	TypeScript execution

3. Project Structure

```

medlicense/
├── src/
│   ├── app/
│   │   ├── (admin)/admin/      # Admin panel pages
│   │   ├── (auth)/             # Login, register, verify-email, forgot-password
│   │   ├── (dashboard)/        # Student dashboard pages
│   │   ├── (public)/           # Landing, pricing, about, blog, FAQ
│   │   ├── api/                # API routes
│   │   ├── globals.css         # Global styles + Tailwind config
│   │   ├── layout.tsx          # Root layout (ThemeProvider, Providers)
│   │   └── providers.tsx       # SessionProvider + LanguageProvider
│   ├── components/
│   │   ├── admin/              # Admin-only components
│   │   ├── dashboard/overview/ # Dashboard widget components
│   │   ├── exam/               # ExamEngine component
│   │   ├── layout/             # Navbar, Sidebars, Header, Notifications
│   │   ├── pages/              # Landing page sections
│   │   └── ui/                 # Reusable UI primitives
│   ├── lib/
│   │   ├── auth.ts             # NextAuth configuration
│   │   ├── supabase.ts         # Supabase client
│   │   ├── mux.ts              # Mux video client
│   │   ├── email.ts            # Email sending (Nodemailer)
│   │   ├── language.tsx        # LanguageProvider + useLanguage hook
│   │   ├── translations.ts     # EN/FR UI string dictionary
│   │   ├── theme.tsx           # ThemeProvider + useTheme hook
│   │   ├── license-categories.ts # 16 license type definitions
│   │   ├── validations.ts      # Zod validation schemas
│   │   └── utils.ts            # Shared helpers
│   └── types/
│       ├── index.ts            # Shared TypeScript types
│       └── next-auth.d.ts      # NextAuth session type extensions
├── supabase/
│   ├── migrations/             # SQL migration files
│   └── seeds/                  # Seed data (categories, demo accounts)
├── public/                     # Static assets
├── DOCUMENTATION.md            # This file
├── CLAUDE.md                   # AI assistant project instructions
└── AGENTS.md                   # AI agent guidance

```

4. Environment Setup

Prerequisites

- Node.js 18+

- pnpm (`npm install -g pnpm`)
- A Supabase project
- Google OAuth credentials (optional)
- Mux account (optional, for videos)
- SMTP server (for emails)

Installation

```
git clone <repo-url>
cd medlicense
pnpm install
cp .env.example .env.local
# Fill in .env.local values (see below)
pnpm dev
```

Environment Variables

```
# — Supabase —
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...

# — NextAuth —
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=any-random-32-char-string

# — Google OAuth (optional) —
GOOGLE_CLIENT_ID=xxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxx

# — Email (SMTP) —
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your@gmail.com
SMTP_PASSWORD=your-app-password
SMTP_FROM="MedLicense <noreply@medlicense.rw>"

# — Mux (video streaming) —
MUX_TOKEN_ID=your-token-id
MUX_TOKEN_SECRET=your-token-secret
MUX_WEBHOOK_SECRET=your-webhook-secret

# — Afripay (payments) —
AFRIPAY_PUBLIC_KEY=pk...
AFRIPAY_SECRET_KEY=sk...
AFRIPAY_WEBHOOK_SECRET=whsec...

# — App —
NEXT_PUBLIC_APP_URL=http://localhost:3000
NEXT_PUBLIC_APP_NAME=MedLicense
```

Database Setup

Run migrations in order:

```
# In Supabase SQL editor or via supabase CLI:
supabase/migrations/001_add_license_category.sql
supabase/migrations/002_add_license_categories.sql
supabase/migrations/003_license_category_on_categories.sql

# Seed data:
supabase/seeds/seed_categories.sql      # 128 subject categories
supabase/seeds/create_demo_doctor.sql   # demo doctor account
supabase/seeds/reset_demo.sql           # reset demo data
```

Demo Accounts

Email	Password	Role	License
admin@demo.com	demo1234	ADMIN	—
doctor@demo.com	demo1234	STUDENT	Medical Doctor
student@demo.com	demo1234	STUDENT	Dentist

5. Database Schema

users

Column	Type	Description
<code>id</code>	UUID PK	Auto-generated
<code>name</code>	TEXT	Full name
<code>email</code>	TEXT UNIQUE	Login email
<code>password</code>	TEXT	bcryptjs <code>\$2b\$</code> hash
<code>phone</code>	TEXT	Optional phone
<code>country</code>	TEXT	Country of residence
<code>role</code>	TEXT	<code>STUDENT</code> or <code>ADMIN</code>
<code>is_banned</code>	BOOLEAN	Account suspension flag
<code>email_verified</code>	TIMESTAMPTZ	NULL = unverified
<code>license_category</code>	TEXT	One of 16 license type IDs
<code>language</code>	ENUM('EN','FR')	Display language preference
<code>image</code>	TEXT	Profile picture (Google OAuth)
<code>last_login_at</code>	TIMESTAMPTZ	Last successful login
<code>created_at</code>	TIMESTAMPTZ	Account creation time

`exams`

Column	Type	Description
<code>id</code>	UUID PK	—
<code>title_en</code>	TEXT	English title
<code>title_fr</code>	TEXT	French title (optional)
<code>description</code>	TEXT	Short description
<code>category_id</code>	UUID FK → categories	Subject category
<code>license_category</code>	TEXT	NULL = all licenses
<code>duration_minutes</code>	INTEGER	Time limit
<code>passing_score</code>	INTEGER	Pass threshold %
<code>total_questions</code>	INTEGER	Target question count
<code>is_free</code>	BOOLEAN	Free or premium
<code>is_published</code>	BOOLEAN	Visible to students
<code>shuffle_questions</code>	BOOLEAN	Randomize order
<code>shuffle_answers</code>	BOOLEAN	Randomize answers
<code>negative_marking</code>	BOOLEAN	−0.25 per wrong answer
<code>created_at</code>	TIMESTAMPTZ	—

`questions`

Column	Type	Description
<code>id</code>	UUID PK	—
<code>text_en</code>	TEXT	Question text (English) — required
<code>text_fr</code>	TEXT	Question text (French) — optional
<code>explanation_en</code>	TEXT	Correct answer explanation (EN)
<code>explanation_fr</code>	TEXT	Correct answer explanation (FR)
<code>difficulty</code>	TEXT	<code>EASY</code> , <code>MEDIUM</code> , or <code>HARD</code>
<code>category_id</code>	UUID FK → categories	Subject category
<code>license_categories</code>	TEXT[]	Array of applicable license type IDs
<code>type</code>	TEXT	<code>MULTIPLE_CHOICE</code> (only type currently)
<code>image_url</code>	TEXT	Optional clinical image
<code>is_approved</code>	BOOLEAN	Admin-approved flag
<code>is_active</code>	BOOLEAN	Soft-delete flag
<code>created_at</code>	TIMESTAMPTZ	—

answers

Column	Type	Description
<code>id</code>	UUID PK	—
<code>question_id</code>	UUID FK → questions	Parent question
<code>text_en</code>	TEXT	Answer text (English)
<code>text_fr</code>	TEXT	Answer text (French) — optional
<code>is_correct</code>	BOOLEAN	Exactly one per question is true
<code>order</code>	INTEGER	Display order (0-based)

categories

Column	Type	Description
<code>id</code>	UUID PK	—
<code>name_en</code>	TEXT	Category name (EN)
<code>name_fr</code>	TEXT	Category name (FR)
<code>slug</code>	TEXT UNIQUE	URL-safe identifier
<code>license_category</code>	TEXT	NULL = shared, otherwise scoped

Slug format: `{license-key}-{subject-slug}` e.g. `md-internal-medicine` , `dentist-oral-anatomy-physiology`

`exam_questions`

Column	Type	Description
<code>exam_id</code>	UUID FK → exams	—
<code>question_id</code>	UUID FK → questions	—
<code>order</code>	INTEGER	Position in exam

`exam_attempts`

Column	Type	Description
<code>id</code>	UUID PK	—
<code>user_id</code>	UUID FK → users	Who took the exam
<code>exam_id</code>	UUID FK → exams	Which exam
<code>status</code>	TEXT	<code>IN_PROGRESS</code> , <code>COMPLETED</code> , <code>TIMED_OUT</code>
<code>score</code>	DECIMAL	Final score % (0–100)
<code>correct</code>	INTEGER	Correct answers count
<code>wrong</code>	INTEGER	Wrong answers count
<code>skipped</code>	INTEGER	Unanswered count
<code>time_taken</code>	INTEGER	Seconds spent
<code>started_at</code>	TIMESTAMPTZ	Exam start time
<code>submitted_at</code>	TIMESTAMPTZ	Submission time
<code>saved_state</code>	JSONB	Auto-save state (answers, flagged, index, timeLeft)

`attempt_answers`

Column	Type	Description
<code>attempt_id</code>	UUID FK → exam_attempts	—
<code>question_id</code>	UUID FK → questions	—
<code>answer_id</code>	UUID FK → answers	Student's chosen answer (NULL = skipped)
<code>is_correct</code>	BOOLEAN	Pre-computed correctness

`subscriptions`

Column	Type	Description
<code>user_id</code>	UUID FK → users PK	One subscription per user
<code>status</code>	TEXT	<code>FREE</code> , <code>TRIAL</code> , <code>ACTIVE</code> , <code>EXPIRED</code>
<code>plan</code>	TEXT	<code>monthly</code> or <code>annual</code>
<code>start_date</code>	DATE	Subscription start
<code>end_date</code>	DATE	Subscription expiry
<code>auto_renew</code>	BOOLEAN	Auto-renewal flag

payments

Column	Type	Description
<code>id</code>	UUID PK	—
<code>user_id</code>	UUID FK → users	—
<code>amount</code>	DECIMAL	Amount in RWF
<code>currency</code>	TEXT	<code>RWF</code>
<code>provider</code>	TEXT	<code>afripay</code> , <code>momo</code> , <code>card</code>
<code>plan</code>	TEXT	<code>monthly</code> or <code>annual</code>
<code>status</code>	TEXT	<code>pending</code> , <code>completed</code> , <code>failed</code>
<code>created_at</code>	TIMESTAMPTZ	—

notifications

Column	Type	Description
<code>id</code>	UUID PK	—
<code>user_id</code>	UUID FK → users	Recipient
<code>title</code>	TEXT	Short heading
<code>message</code>	TEXT	Notification body
<code>type</code>	TEXT	<code>info</code> , <code>success</code> , <code>warning</code> , <code>error</code>
<code>is_read</code>	BOOLEAN	Read status
<code>created_at</code>	TIMESTAMPTZ	—

saved_questions

Column	Type	Description
user_id	UUID FK → users	—
question_id	UUID FK → questions	—
saved_at	TIMESTAMPTZ	—

daily_streaks

Column	Type	Description
user_id	UUID FK → users PK	—
current_streak	INTEGER	Consecutive days
longest_streak	INTEGER	All-time best
points	INTEGER	Gamification points
last_activity_date	DATE	Last logged day

videos

Column	Type	Description
id	UUID PK	—
title_en	TEXT	Video title (EN)
description	TEXT	Optional description
category_id	UUID FK → categories	Subject area
mux_asset_id	TEXT	Mux asset ID (after processing)
mux_playback_id	TEXT	Mux playback ID
duration	INTEGER	Video length in seconds
thumbnail	TEXT	Thumbnail image URL
is_free	BOOLEAN	Free or premium
is_published	BOOLEAN	Visible to students
created_at	TIMESTAMPTZ	—

6. Authentication

Authentication uses **NextAuth v5 (JWT strategy)** with two providers:

Credentials (Email + Password)

1. User submits email + password
2. Supabase query fetches user record
3. `bcryptjs.compare()` validates the password (`$2b$` format)
4. Checks: `is_banned` , `email_verified`
5. Updates `last_login_at`
6. Returns user object → JWT token

Google OAuth

1. Google returns user profile
2. If new user → auto-creates account in Supabase with `email_verified = now()`
3. Existing user → fetches `role` , `license_category` , `language` from DB
4. Returns JWT token

JWT Token Payload

```
{
  id: string;
  email: string;
  name: string;
  role: "STUDENT" | "ADMIN";
  licenseCategory: string | null; // e.g. "medical_doctor"
  language: "EN" | "FR";
}
```

Session Object

```
session.user = {
  id: string;
  email: string;
  name: string;
  role: string;
  licenseCategory: string | null;
  language: string | null;
}
```

Email Verification Flow

1. Registration → generates 64-char hex token → stores in `verification_tokens` (24h expiry)
2. Sends email via SMTP with `/api/auth/verify-email?token=X`
3. On click → marks `users.email_verified = now()` → deletes token → redirects to login

Password Reset Flow

1. POST `/api/auth/forgot-password` → generates 64-char token (1h expiry)
 2. Sends reset link email
 3. User clicks link → sets new password → token deleted
-

7. API Reference

Auth Endpoints

POST `/api/auth/register`

Registers a new student account.

Request body:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "+250700000000",
  "password": "securepass123",
  "licenseCategory": "medical_doctor"
}
```

Response: `{ message: "Verification email sent" }` or `{ error: "..." }`

GET `/api/auth/verify-email?token=<token>`

Verifies email from the link sent during registration.

POST `/api/auth/forgot-password`

Sends a password reset email. Always returns 200 (silent on unknown email).

Request body: `{ "email": "john@example.com" }`

User Endpoints

PATCH `/api/users/profile`

Updates the logged-in user's profile.

Request body:

```
{
  "name": "John Doe",
  "phone": "+250700000000",
  "country": "Rwanda",
  "language": "FR"
}
```

PATCH /api/users/language

Updates only the language preference. Lightweight endpoint used by the language toggle.

Request body: { "language": "EN" | "FR" }

Exam Endpoints

PATCH /api/exams/[attemptId]/save

Auto-saves exam progress every 30 seconds.

Request body:

```
{
  "answers": { "questionId": "answerId" },
  "flagged": { "questionId": true },
  "currentIndex": 5,
  "timeLeft": 1500
}
```

POST /api/exams/[attemptId]/submit

Submits the exam, computes score, and saves all answers.

Request body:

```
{
  "answers": { "questionId": "answerId" },
  "timedOut": false
}
```

Response: { score, correct, wrong, skipped, passed }

Scoring logic:

- Base: $(\text{correct} / \text{total}) * 100$
 - If `negative_marking = true`: `score -= (wrong * 0.25)`
-

Notification Endpoints

GET /api/notifications

Returns the 30 most recent notifications for the current user.

PATCH /api/notifications

Marks all unread notifications as read.

PATCH /api/notifications/[id]

Marks a single notification as read.

DELETE /api/notifications/[id]

Deletes a single notification.

Payment Endpoints

POST /api/payments/initiate

Creates a pending payment and returns redirect URLs.

Request body:

```
{
  "plan": "monthly",
  "provider": "momo"
}
```

Response: { paymentId, redirectUrl }

Pricing:

Plan	Price
Monthly	15,000 RWF
Annual	120,000 RWF

GET /api/payments/callback?paymentId=X&status=completed

Payment provider redirects here after payment. Activates subscription on success.

Admin: Questions

POST /api/admin/questions

Creates a new question with multilingual support.

Request body:

```
{
  "text_en": "A 45-year-old patient...",
  "text_fr": "Un patient de 45 ans...",
  "explanation_en": "The correct answer is...",
  "explanation_fr": "La bonne réponse est...",
  "difficulty": "MEDIUM",
  "category_id": "uuid",
  "license_categories": ["medical_doctor", "specialist_doctor"],
  "answers": [
    { "text_en": "Option A", "text_fr": "Option A en FR", "is_correct": true },
    { "text_en": "Option B", "text_fr": "Option B en FR", "is_correct": false }
  ]
}
```

PATCH /api/admin/questions/[id]

Updates question and replaces all answers.

DELETE /api/admin/questions/[id]

Permanently deletes question and its answers.

POST /api/admin/questions/csv

Bulk imports questions from a CSV file.

Required CSV columns:

Column	Description
category_slug	Must match an existing categories.slug
difficulty	EASY , MEDIUM , or HARD
text_en	Question text
explanation_en	Optional explanation
answer_a through answer_d	Answer option texts
correct_letter	A , B , C , or D
license_categories	Comma-separated license IDs (e.g. md-internal-medicine)

Admin: Exams

POST /api/admin/exams

Creates an exam and links a list of question IDs.

Request body:

```
{
  "title_en": "Medical Doctor Mock Exam 1",
  "title_fr": "Examen simulé Médecin 1",
  "category_id": "uuid",
  "license_category": "medical_doctor",
  "duration_minutes": 90,
  "passing_score": 60,
  "is_free": false,
  "shuffle_questions": true,
  "shuffle_answers": true,
  "negative_marking": false,
  "questionIds": ["uuid1", "uuid2"]
}
```

POST /api/admin/exams/generate-questions

Smart question selection for exam building.

Request body:

```
{
  "categoryBreakdown": [
    { "catId": "uuid", "count": 10 }
  ],
  "difficulty": "balanced",
  "licenseCategory": "medical_doctor"
}
```

Difficulty modes:

Mode	Easy	Medium	Hard
balanced	30%	50%	20%
progressive	20%	50%	30%
challenge	10%	40%	50%

PATCH /api/admin/exams/[examId]

Update exam details or toggle `is_published`.

DELETE /api/admin/exams/[examId]

Delete exam and unlink all questions.

Admin: Videos

POST /api/admin/videos/upload-url

Returns a Mux direct upload URL and creates a video DB record.

PATCH /api/admin/videos/[videoId]

Updates video title, description, category, free/published flags.

DELETE /api/admin/videos/[videoId]

Deletes video record.

Admin: Users

PATCH /api/admin/users/[userId]

Admin actions on a user account.

Request body: { "action": "ban" | "unban" | "make_admin" | "make_student" }

Admin: Notifications

POST /api/admin/notifications

Send a notification to one user or broadcast to all students.

Request body:

```
{
  "user_id": "uuid",           // optional - omit to broadcast to all students
  "title": "New exam added",
  "message": "A new exam is now available...",
  "type": "info"               // info | success | warning | error
}
```

Webhooks

POST /api/webhooks/mux

Mux sends events here after video processing.

- video.asset.ready → updates mux_asset_id, mux_playback_id, duration, thumbnail
- video.asset.error → logs error, no DB change

8. Pages & Routing

Public Pages ((public)/)

Route	Page
/	Landing page (Hero, Features, Stats, Pricing preview, Testimonials, CTA)
/about	About MedLicense
/pricing	Full pricing plans
/free-trial	Free trial signup
/blog	Blog articles listing
/tutorials	Video tutorials
/contact	Contact form
/faq	Frequently Asked Questions

Auth Pages ((auth)/)

Route	Page
/login	Email/password + Google OAuth login
/register	New account registration
/verify-email	Email verification landing
/forgot-password	Password reset request

Student Dashboard ((dashboard)/)

All routes require authentication (redirect to /login if not signed in).

Route	Page	Description
/dashboard	Overview	Stats, readiness gauge, recent attempts, leaderboard
/exams	Exam list	All published exams for the user's license category
/exams/[id]	Exam taker	Launches the ExamEngine for the selected exam
/results	Results list	All completed attempts with scores
/results/[attemptId]	Result detail	Per-question breakdown with correct answers
/analytics	Analytics	Performance charts by category, score progression
/saved	Saved questions	Bookmarked questions from past exams
/subscription	Subscription	Current plan + upgrade options
/profile	Profile	Name, phone, country, language, license category

Admin Panel ((admin)/admin/)

All routes require `role === "ADMIN"`.

Route	Page
/admin	KPI dashboard
/admin/questions	Question management
/admin/exams	Exam management
/admin/users	User management
/admin/analytics	Platform analytics
/admin/payments	Payment management
/admin/content	Video content management

9. Core Features

Exam Readiness Score

Computed as a weighted formula on the student's history:

```
readiness = avgScore × 0.6 + passRate × 0.3 + min(totalExams × 5, 10)
```

Capped at 100. Shown on the dashboard as a visual gauge.

Auto-Save

Every 30 seconds during an active exam, the client sends a PATCH to `/api/exams/[attemptId]/save` with current state (answers, flagged questions, current index, time remaining). Saved to `exam_attempts.saved_state` as JSONB.

Negative Marking

If `exam.negative_marking = true`, the final score is reduced by 0.25 points per wrong answer:

```
rawScore = (correct / total) * 100
finalScore = rawScore - (wrong * 0.25)
```

Study Streak

Tracked in `daily_streaks` per user. Increments `current_streak` when a user completes an exam on a new calendar day. Resets to 0 if a day is missed.

License-Specific Question Filtering

Questions store a `license_categories TEXT[]` array. The generate-questions API filters with:

```
WHERE license_categories @> ARRAY['medical_doctor']
```

Exams set `license_category` to scope them to a user's profession. Student exam lists filter:

```
WHERE license_category = 'medical_doctor' OR license_category IS NULL
```

10. Admin Panel

Question Management

- **Create:** Fill EN + FR tabs, assign difficulty, subject category, license types, add 2–6 answer options
- **Edit:** Pre-populated form, replace answers on save
- **Delete:** Hard delete (cascades to answers)
- **Approve:** Toggle `is_approved` flag
- **Filter:** By subject category, approval status
- **CSV Import:** Bulk upload with template download

FR badge on question cards:

- FR FR badge = French translation exists
- "EN only" badge = no French version yet

Exam Builder

- **Manual mode:** Pick individual questions
- **Smart Build mode:** Define a per-category count, choose a difficulty profile (balanced/progressive/challenge), generate automatically
- **Settings:** Duration, passing score, free/premium, shuffle options, negative marking, license scope

User Management

- View all students with last login, exam count, subscription status
- Actions: Ban, Unban, Promote to Admin, Demote to Student

Broadcast Notifications

Send system notifications to one user or all students via the admin panel or POST

`/api/admin/notifications`.

Analytics Dashboard

- Revenue per month
 - Subscriptions breakdown (active/trial/expired)
 - Top-performing students
 - Most-taken exams
-

11. Exam Engine

The exam engine (`src/components/exam/ExamEngine.tsx`) is a self-contained client component that handles the entire exam-taking experience.

Features

- **Timer** — Countdown with color transitions: normal → warning (under 10 min) → urgent (under 5 min, pulsing)
- **Progress bar** — Percentage of questions answered
- **Two-column answer grid** — A/B on left, C/D on right
- **Flag button** — Mark questions for review; flagged questions show amber in navigator
- **Question navigator sidebar** — Grid of all question numbers, color-coded by status
- **Auto-save** — Every 30 seconds
- **Submit confirmation modal** — Shows answered/unanswered/flagged counts before final submit
- **Timeout modal** — Auto-submits when time expires
- **Bilingual content** — Renders `text_fr` / `textFr` when the user's language is FR

Question Navigator Colors

Color	Meaning
Blue	Current question
Green	Answered
Amber	Flagged
Gray	Not yet answered

Answer Selection

Clicking an answer circle selects it and shows a blue highlight + checkmark. Clicking another answer replaces the selection. No deselect (must skip by leaving unanswered).

12. Localization (EN/FR)

Architecture

```

LanguageProvider (src/lib/language.tsx)
└─ reads localStorage on mount
└─ overrides with session.user.language on first auth
└─ persists changes to localStorage + DB (/api/users/language)

useLanguage() hook → { language, setLanguage }
useT(language) → T("key") function from translations.ts
t(en, fr, language) → picks correct string for bilingual DB content

```

Language Toggle

Located in `DashboardHeader.tsx`. Shows current inactive language as the button label (e.g. shows "FR" when currently viewing EN). One click switches instantly across all client components.

What Gets Translated

Static UI (translations.ts dictionary):

- Sidebar navigation labels
- Dashboard card titles and stat labels
- Exam listing page
- Results and analytics pages
- Saved questions page
- Exam engine (all labels, modals, navigator)
- Notifications dropdown
- Quick Actions widget

- Welcome Banner (including motivational quotes in both languages)

Dynamic DB content (using `t(en, fr, lang)` helper):

- Exam titles (`title_en` / `title_fr`)
- Question text (`text_en` / `text_fr`)
- Answer text (`text_en` / `text_fr`)
- Question explanations (`explanation_en` / `explanation_fr`)
- Category names (`name_en` / `name_fr`)

Adding a New Translation Key

1. Add to `src/lib/translations.ts` :

```
my_new_key: { EN: "English text", FR: "Texte en français" },
```

2. Use in a client component:

```
const { language } = useLanguage();
const T = useT(language);
// ...
<p>{T("my_new_key")}</p>
```

3. Use for bilingual DB content:

```
import { t } from "@lib/language";
t(item.title_en, item.title_fr, language)
```

13. Payments & Subscriptions

Plans

Plan	Price	Duration
Premium Monthly	15,000 RWF	30 days
Premium Annual	120,000 RWF	365 days (saves ~33%)

Flow

```
Student clicks "Upgrade"
  → POST /api/payments/initiate (creates pending payment)
  → Redirect to Afripay/MoMo/Card provider
  → Provider redirects to /api/payments/callback?status=completed
  → Subscription activated (status = ACTIVE, end_date set)
```

Subscription Status Values

Status	Description
FREE	No active subscription (default)
TRIAL	Free trial period active
ACTIVE	Paid subscription active
EXPIRED	Past subscription, not renewed

Premium Content Gating

Exams with `is_free = false` show a lock icon and "Upgrade to Unlock" button to students without `ACTIVE` or `TRIAL` subscriptions.

14. Video Tutorials (Mux)

Upload Flow

```
Admin clicks "Add Video"
  → POST /api/admin/videos/upload-url
    → Creates DB record (status: pending)
    → Mux returns direct upload URL
  → Frontend uploads video file directly to Mux
  → Mux processes the video asynchronously
  → Mux fires webhook: POST /api/webhooks/mux
    → video.asset.ready → saves asset_id, playback_id, duration, thumbnail
```

Playback

Uses `@mux/mux-player-react` component with the `mux_playback_id`. Free videos are accessible to all authenticated users; premium videos require `ACTIVE` or `TRIAL` subscription.

15. Notifications

Notification Types

Type	Icon	Color
info	Info circle	Blue
success	Check circle	Emerald
warning	Alert triangle	Amber
error	Alert circle	Red

Bell Dropdown

- Auto-refreshes every 60 seconds
- Shows red badge with unread count (up to "9+")
- Unread notifications have blue background + blue dot
- Click on an unread notification marks it as read
- X button deletes individual notifications
- "Mark all read" button clears the entire unread count

Sending Notifications (Admin)

```
POST /api/admin/notifications
{
  "title": "New exam available",
  "message": "A new Medical Doctor mock exam is ready for practice.",
  "type": "success"
  # omit user_id to broadcast to all non-banned students
}
```

16. License Categories

The platform supports **16 license types** across **5 professional groups**:

Medical

ID	Label
medical_doctor	Medical Doctor (General)
specialist_doctor	Specialist Doctor

Dental

ID	Label
dentist	Dentist / Dental Surgeon
dental_technician	Dental Technician

Pharmacy

ID	Label
pharmacist	Pharmacist
pharmacy_technician	Pharmacy Technician

Nursing & Midwifery

ID	Label
nurse_a0	Nurse — A0 (Bachelor)
nurse_a1	Nurse — A1 (Advanced Diploma)
nurse_a2	Nurse — A2 (Certificate)
midwife	Midwife

Allied Health

ID	Label
physiotherapist	Physiotherapist
lab_technician	Laboratory Technician
radiology_technician	Radiology Technician
nutritionist	Nutritionist / Dietitian
env_health	Environmental Health Officer
orthopedic_technician	Orthopedic Technician

Subject Categories

Each license type has **8 subject categories** seeded (128 total). Category slugs follow the pattern:

{license-key}-{subject-slug} (e.g. md-internal-medicine , dentist-oral-anatomy-physiology).

17. Components Reference

Layout Components

DashboardSidebar

File: `src/components/layout/DashboardSidebar.tsx` Student navigation sidebar. Auto-translates navigation labels based on current language. Highlights active route.

DashboardHeader

File: `src/components/layout/DashboardHeader.tsx` Top bar with page title/subtitle, language toggle (EN↔FR), theme toggle (dark/light), notifications bell, and user avatar.

NotificationsDropdown

File: `src/components/layout/NotificationsDropdown.tsx` Bell icon + dropdown panel. Polls every 60 seconds. Supports mark-as-read and delete. Fully bilingual.

Dashboard Widgets

WelcomeBanner

File: `src/components/dashboard/overview/WelcomeBanner.tsx` Gradient banner with personalized greeting (morning/afternoon/evening), streak badge, license label, and motivational quote (rotated daily, available in EN and FR per profession).

ReadinessGauge

File: `src/components/dashboard/overview/ReadinessGauge.tsx` Circular gauge showing exam readiness score with color coding.

QuickActions

File: `src/components/dashboard/overview/QuickActions.tsx` 6-tile grid: Daily Quiz, Mock Exam, Saved, Tutorials, Analytics, Subscription/Premium.

StudyStreakCard

File: `src/components/dashboard/overview/StudyStreakCard.tsx` Shows current streak, longest streak, points, and badge count.

TopPerformers

File: `src/components/dashboard/overview/TopPerformers.tsx` Leaderboard of top 5 students this month.

RecentActivity

File: `src/components/dashboard/overview/RecentActivity.tsx` Last 5 exam attempts with scores and pass/fail badges.

Admin Components

QuestionFormModal

File: `src/components/admin/QuestionFormModal.tsx` Full question create/edit modal with:

- EN/FR language tab switcher
- License category multi-selector (filters subject categories)
- Subject category dropdown (filtered by license selection)
- Difficulty picker
- 2–6 answer options with correct answer marker
- Explanation field
- FR completion indicator in footer

`ExamFormModal`

File: `src/components/admin/ExamFormModal.tsx` Exam create/edit modal with Smart Build support. Filters categories and questions by license category.

`CSVUploadModal`

File: `src/components/admin/CSVUploadModal.tsx` CSV bulk import with template download and upload progress.

`VideoUploadModal`

File: `src/components/admin/VideoUploadModal.tsx` Mux direct-upload modal with drag-and-drop.

Exam Component

`ExamEngine`

File: `src/components/exam/ExamEngine.tsx` Full exam experience. Accepts a pre-built `ExamData` object and manages all state locally. Connects to save and submit APIs. Renders question text and answers in the user's active language.

UI Primitives (`src/components/ui/`)

Component	Description
<code>Badge</code>	Status chips (variants: default, success, danger, warning, info)
<code>Button</code>	Styled button (variants: default, outline, ghost; sizes: default, sm, lg)
<code>Card</code> / <code>CardContent</code> / <code>CardHeader</code> / <code>CardTitle</code>	Container cards
<code>Input</code>	Form input with label and error state
<code>Modal</code>	Generic dialog overlay
<code>Progress</code>	Horizontal progress bar with optional label

18. Deployment

Vercel (Recommended)

```
# Install Vercel CLI
npm i -g vercel

# Deploy
vercel
```

Required Vercel settings:

- Framework: Next.js
- Build Command: `pnpm build`
- Output Directory: `.next`
- Node.js Version: 18.x

Environment Variables: Set all variables from Section 4 in the Vercel dashboard.

Important: The `NEXTAUTH_URL` must be set to your production domain (e.g. `https://medlicense.rw`).

Mux Webhook Configuration

After deploying, register the webhook endpoint in the Mux dashboard:

```
https://medlicense.rw/api/webhooks/mux
```

Supabase RLS Policies

Ensure Row Level Security policies are configured in Supabase to match how the service role key is used server-side. Student-facing queries use the service role key (bypasses RLS) on the server — do not expose the service role key to the client.

Production Checklist

- ☐ All `.env.local` values set in Vercel environment variables
 - ☐ Database migrations applied to production Supabase project
 - ☐ Seed categories imported (`supabase/seeds/seed_categories.sql`)
 - ☐ Mux webhook registered and verified
 - ☐ Google OAuth redirect URIs updated to production domain
 - ☐ SMTP email sender verified
 - ☐ `NEXTAUTH_SECRET` set to a strong random value
 - ☐ Demo accounts created (or removed for production)
-

Documentation last updated: May 2026 Platform: MedLicense v1.0 — Built for Rwanda's healthcare professionals